

Valor 5% Tarea #2 – Trabajar en Clase, 1 hora y 45 minutos

PASO A PASO DE LA ACTIVIDAD

PASO 1 – Entender el sistema

De acuerdo al caso asignado responda:

1. ¿Cuál es el objetivo principal del sistema?
2. ¿Quiénes son los usuarios?
3. ¿Qué datos procesa la aplicación?
4. Escriba 3 suposiciones técnicas sobre lo que *no* dice el texto de la descripción de la aplicación, debe ser consistente con el resto de la descripción proporcionada

Resultado esperado

Una descripción corta del sistema (máx. 10 líneas).

PASO 2 – Identificar activos críticos

Instrucciones

Identifica 5 activos que necesitan protección.

Ejemplos:

- Datos personales
- Credenciales
- Información de pago
- Reputación del negocio

Usar la siguiente tabla sugerida

Activo	Tipo	¿Por qué es crítico?

Guía:

Qué pasaría si ese activo se pierde, se altera o se filtra.

Responda:

Si solo tienes presupuesto para proteger dos activos de la tabla, ¿cuáles priorizas y por qué descarta los otros dos?

PASO 3 – Dibujar el flujo de datos

Instrucciones

Dibuja a mano y pega la foto de la imagen en el documento un **diagrama simple** que muestre:

- Usuario

- Servidor
- Base de datos
- Servicios externos

Marca con color rojo en el diagrama los **límites de confianza**.

No se evalúa estética, se evalúa comprensión.

PASO 4 – Identificar amenazas

Instrucciones

¿Qué podría salir mal aquí?

Identifica 2 amenazas donde el atacante sea un empleado interno con permisos válidos y
Identifica una amenaza donde el vector de ataque sea el servicio externo de terceros.
Agrega otras 2 amenazas, en total la tabla debe tener 5 amenazas.

Tabla sugerida

Amenaza	Activo afectado

Ejemplo:

Amenaza: Robo de credenciales

Activo: Cuenta del usuario

Responda:

PASO 5 – Evaluar riesgos

Instrucciones

Evaluar cada amenaza según:

- Impacto (Alto / Medio / Bajo)
- Probabilidad (Alta / Media / Baja)

Tabla sugerida

Amenaza	Impacto	Probabilidad	Riesgo

Guía:

No todas las amenazas son igual de graves.

PASO 6 – Definir mitigaciones desde el diseño

Instrucciones

Proponer **medidas de diseño**, no parches de código.

Ejemplos:

- Minimización de datos
- Consentimiento explícito
- Cifrado
- Autenticación fuerte
- Separación de roles

Tabla sugerida

Amenaza	Mitigación desde diseño

Pregunta clave:

¿Cómo evitarías este problema **antes de programar**?

ENTREGABLE FINAL

Un documento corto (2–5 páginas máximo) que incluya la respuesta de cada una de los pasos anteriores

“El Threat Modeling no es buscar errores: es diseñar sistemas responsables.”

RUBRICA DE EVALUACIÓN

Criterio	4 – Excelente	3 – Bueno	2 – Básico	1 – Insuficiente
1. Comprensión del sistema	Describe claramente el propósito del sistema, usuarios y tipos de datos tratados.	Describe el sistema con pequeños vacíos.	Descripción incompleta o confusa.	No demuestra comprensión del sistema.
2. Identificación de activos críticos	Identifica activos relevantes y justifica claramente por qué son críticos.	Identifica activos con justificación parcial.	Lista activos sin justificar su criticidad.	No identifica activos críticos.
3. Diagrama de Flujo de Datos (DFD)	Diagrama claro con componentes, flujos y límites de confianza bien definidos.	Diagrama entendible pero incompleto.	Diagrama confuso o sin límites de confianza.	No presenta diagrama.
4. Identificación de amenazas	Amenazas relevantes, bien razonadas y alineadas al diseño del sistema.	Amenazas correctas pero limitadas.	Amenazas genéricas o poco relacionadas.	No identifica amenazas reales.

Criterio	4 – Excelente	3 – Bueno	2 – Básico	1 – Insuficiente
5. Evaluación de riesgos (Impacto y Probabilidad)	Prioriza correctamente amenazas según impacto y probabilidad.	Evaluación adecuada con algunos errores.	Evaluación superficial o incompleta.	No evalúa riesgos.
6. Mitigaciones desde el diseño	Propone mitigaciones claras, realistas y aplicables antes del desarrollo.	Mitigaciones adecuadas pero incompletas.	Mitigaciones genéricas o técnicas sin enfoque de diseño.	No propone mitigaciones.
7. Claridad y coherencia del entregable	Documento claro, estructurado y coherente, con 5 páginas como máximo.	Entregable entendible con leves problemas.	Documento desordenado o poco claro.	Entregable incompleto o confuso y más de 5 páginas.

Casos a seleccionar:

1. MedCare Connect (Telemedicina) / GRUPO 1

Plataforma móvil para consultas médicas virtuales. Para acelerar la carga de la ficha médica en zonas con mala señal, la app móvil descarga en la memoria caché del teléfono el historial clínico completo del paciente al iniciar sesión. El médico se autentica con usuario y contraseña mediante una API REST en la nube, pero si el médico pierde la conexión, la app cambia a "modo offline" y valida el acceso comparando la contraseña ingresada contra un hash MD5 guardado localmente en el dispositivo. Las recetas emitidas se generan en PDF y se envían por correo electrónico simple (sin cifrar ni firmar) a la farmacia elegida por el paciente.

2. AutoPark Express (Estacionamientos Inteligentes) / GRUPO 2

Sistema IoT para reservas de estacionamiento. Para reducir la latencia al abrir la barrera, la cámara ALPR de la entrada no consulta al servidor en la nube; en su lugar, el servidor sincroniza cada 5 minutos un archivo JSON con la lista completa de placas y nombres de usuarios hacia la memoria local de la cámara. La cámara almacena los registros de entrada en una base de datos SQLite sin contraseña dentro del dispositivo físico. Además, la app móvil del usuario realiza el cobro enviando el número completo de tarjeta de crédito y el CVV en el cuerpo del request HTTP directamente hacia la API de la pasarela de pagos.

3. EduGrade Cloud (Portal Académico) / GRUPO 3

Plataforma SaaS para gestión de notas. El portal utiliza autenticación SSO mediante Azure AD, pero para mantener compatibilidad con un sistema legado de admisiones, guarda una copia local de la contraseña del usuario en texto plano dentro de la base de datos MySQL del colegio. Cuando un docente sube las calificaciones, la aplicación web procesa el archivo Excel adjunto ejecutando un script en el servidor con permisos de administrador (`root`). Para facilitar el trabajo de los profesores, la sesión del portal no expira por inactividad y permite múltiples inicios de sesión simultáneos con las mismas credenciales desde diferentes IP.

4. PayQuick (Billetera Digital P2P) / GRUPO 4

Aplicación móvil de transferencias instantáneas. Para evitar depender constantemente de la API del banco, la app guarda en el almacenamiento del dispositivo un saldo virtual local que se actualiza antes de confirmar la transacción con el servidor. Cuando un usuario escanea un código QR para pagar, el código contiene directamente la instrucción SQL o la URL de destino que la app ejecuta para realizar la transferencia. La validación del PIN de 6 dígitos para autorizar montos altos se realiza de forma local dentro del código JavaScript/React Native de la aplicación antes de enviar el comando de pago.

5. SmartHome Hub (Gestión IoT Hogareño) / GRUPO 5

Ecosistema en la nube para control de cerraduras y cámaras inteligentes. El gateway físico instalado en la casa se comunica con el servidor en la nube mediante el protocolo MQTT utilizando credenciales globales por defecto fijadas en el firmware (`admin/admin`). Cuando el usuario quiere ver el video en vivo de su cámara, la app móvil solicita al servidor en la nube una URL pública de streaming en un bucket S3 que no requiere tokens de autenticación para ser visualizada. Adicionalmente, las actualizaciones de firmware del gateway se descargan desde un servidor FTP sin cifrado y no incluyen firma digital para verificar su integridad.

6. QuickEat (Delivery de Alimentos) / GRUPO 6 Y GRUPO 12

Plataforma de entrega de comida a domicilio. Para facilitar la comunicación entre el repartidor y el cliente, la app expone en el mapa en tiempo real las coordenadas GPS exactas del repartidor y la dirección física completa del cliente a través de un WebSocket no autenticado. El pago se procesa guardando los tokens de la tarjeta de crédito en el *LocalStorage* del navegador web del cliente. Además, cuando el restaurante confirma un pedido, el panel web del restaurante emite una petición POST a la API donde el parámetro `monto_total` es enviado por el cliente y el servidor lo procesa sin revalidar los precios del menú.

7. SupplyChain Tracker (Trazabilidad de Medicamentos) / GRUPO 7

Sistema web para monitoreo de la cadena de frío de medicamentos. Los sensores de temperatura transmiten telemetría por red celular mediante peticiones HTTP GET, incluyendo la clave de la API de la empresa como un parámetro visible en la URL (`?api_key=SECRET123`). Los operadores de bodega utilizan terminales portátiles con un sistema operativo Android desactualizado (versión 7) que comparte una única cuenta corporativa genérica para todos los empleados. Si un sensor detecta una pérdida de cadena de frío, la alerta se almacena en una base de datos MongoDB abierta a Internet sin control de acceso por roles (RBAC).

8. VoteOnline (Votación Institucional) / GRUPO 8 Y GRUPO 11

Sistema web de elecciones remotas para colegios profesionales. El votante recibe un token de un solo uso por correo electrónico, pero el token se genera utilizando el número de cédula del usuario pasado por una función Hash SHA-1 sin sal (*salt*), lo que permite predecir los tokens de otros votantes. Cuando el usuario vota, el sistema almacena en la misma tabla de la base de datos el ID del votante junto a la opción elegida para "evitar que vote dos veces". Para agilizar la auditoría, la base de datos permite consultas directas vía web a través de un panel de administración protegido únicamente por una cookie de sesión sin flag `HttpOnly`.

9. HR-Core (Recursos Humanos y Planilla) / GRUPO 9

Plataforma en la nube para gestión de nómina. Para automatizar el pago de salarios, el sistema genera quincenalmente un archivo plano con los nombres, números de cuenta bancaria y montos a pagar, y lo deposita en un servidor SFTP compartido utilizando una clave privada SSH almacenada en el repositorio de código fuente de la aplicación. Los empleados pueden subir sus comprobantes de incapacidad médica en formato PDF, pero el servidor guarda los archivos en un directorio público accesible por URL sin verificar si el usuario que consulta es el dueño del documento o parte del equipo de RRHH.

10. TicketPass (Boletos para Eventos Masivos) / GRUPO 10

Plataforma de venta de entradas para conciertos. Para evitar caídas por alto tráfico durante las filas virtuales, la lógica de asignación y reserva de asientos se ejecuta enteramente en el lado del cliente (frontend con JavaScript). Al comprar un boleto, la app genera un código QR que contiene el objeto JSON del boleto (ID de evento, asiento y nombre del comprador) cifrado con una clave simétrica codificada directamente dentro del código ejecutable de la app móvil (*hardcoded*). Al ingresar al estadio, los escáneres validan el código QR de forma offline descifrando el contenido con esa misma clave local.